



HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

The background of the entire image is a dark blue field filled with numerous red dots of varying sizes. These dots are arranged in a pattern that suggests a stylized, abstract representation of a globe or a network, with some areas being denser than others. The dots are scattered across the entire frame, creating a textured, digital feel.

SOICT

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Software Quality Assurance and Testing

6 – Software Testing Process

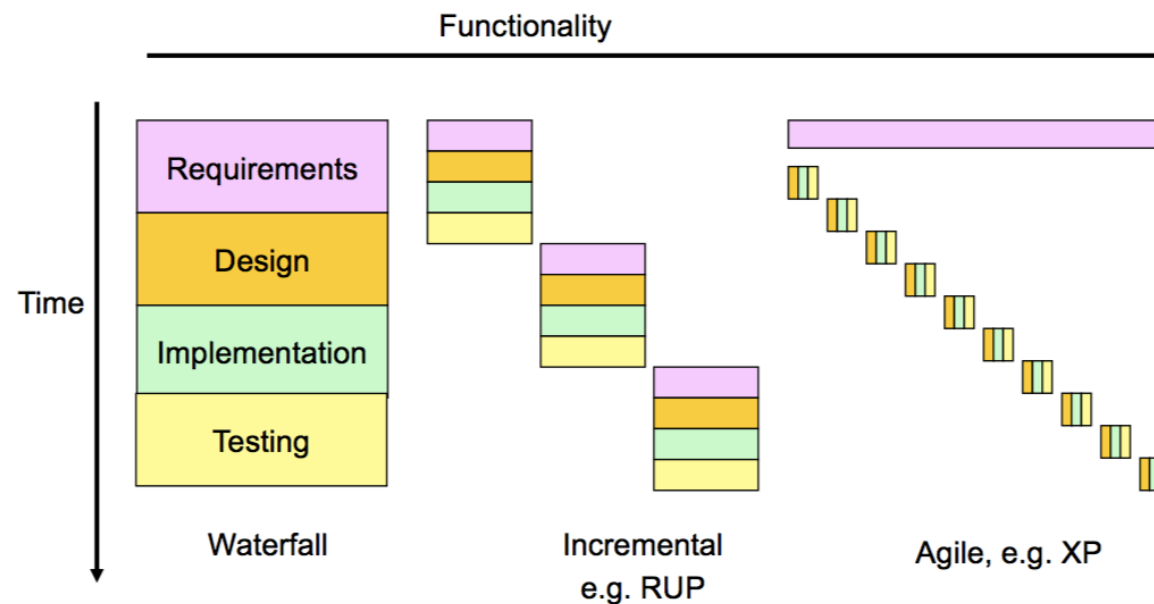
ONE LOVE. ONE FUTURE.

Contents

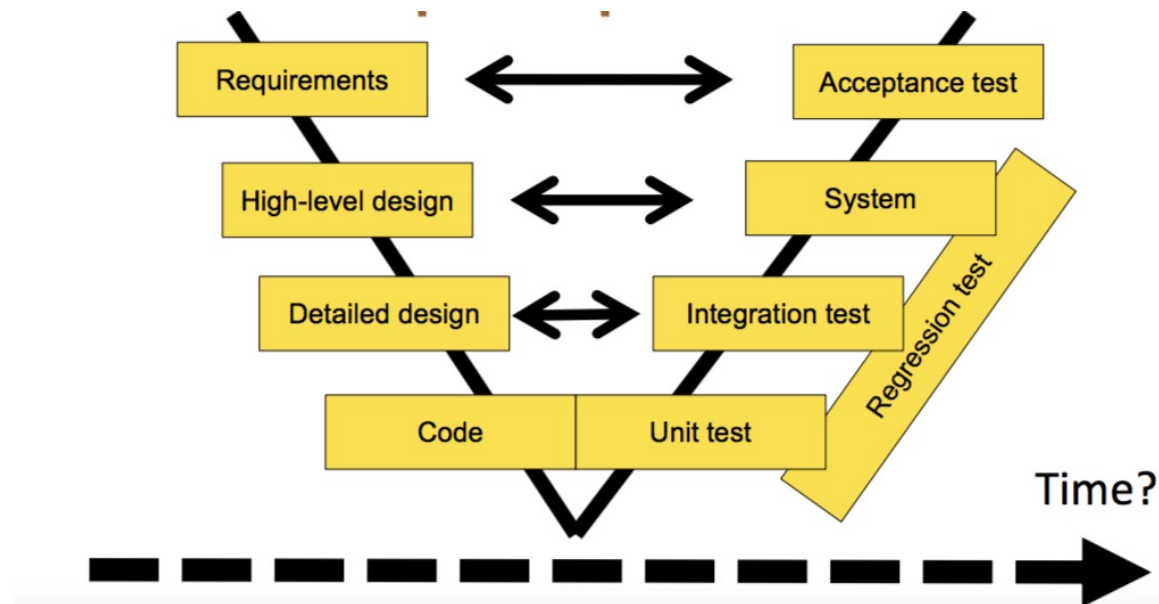
- Software Testing Lifecycle
- System Test Design
- Integration Test
- Regression Test
- Acceptance Test
- Planning and Organization



Process models from waterfall to agile



Phases of testing in the development process



Unit Testing

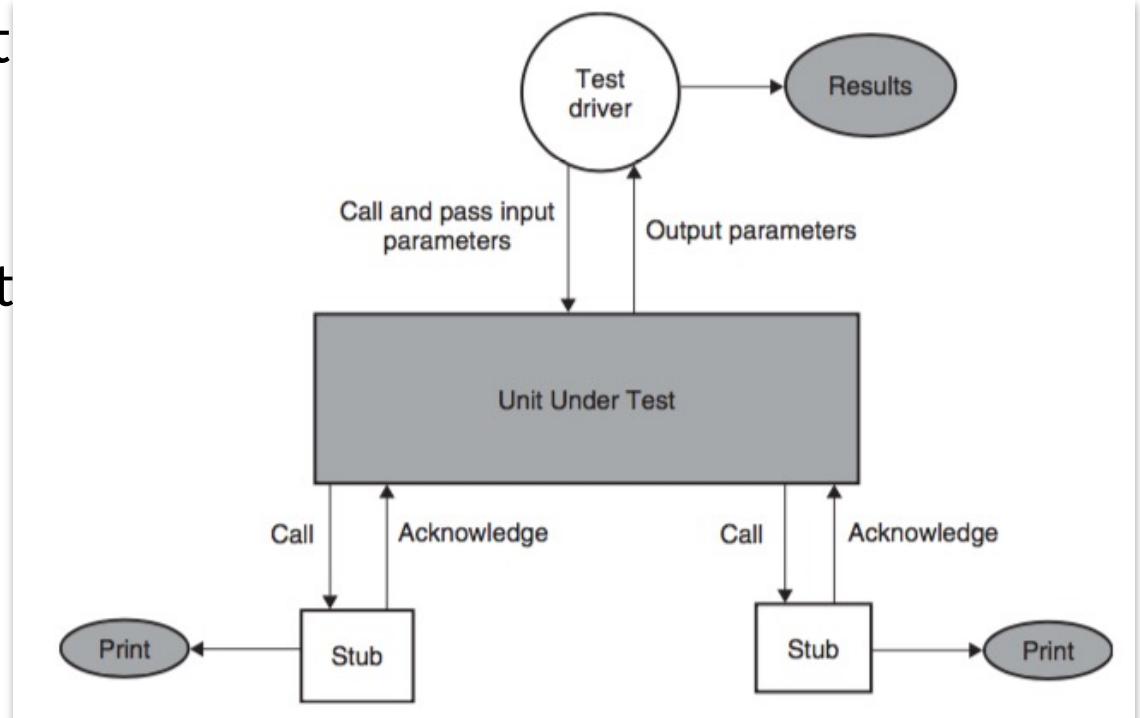
Kiểm thử đơn vị

- Unit = smallest testable software component
 - Objects
 - Procedures/Functions
 - Reusable components
- Tested in isolation
- Usually done by programmer during development
 - A pair tester can help
 - Developer guide or coding guideline can require
- Also known as component or module testing



Dynamic Unit Test Environment

- An environment for dynamic unit testing is created by emulating the context of the unit under test
- The context consists of:
 - A caller of the unit
 - All the other units called by the unit



Test Driver

- The caller unit is called *test driver*
 - A program that invokes the unit under test
 - The unit under test executes with input values received from the driver and return a value to the driver
 - The driver compares the actual outcome with the expected outcome and reports the ensuing test result



Stubs

- The units called by the unit under test are called *stubs*
 - A stub is a *dummy subprogram* that replaces a unit that is called by the unit under test
- A stub performs two tasks:
 - It shows an evidence that the stub was called
 - It returns a precomputed value to the caller so that the unit under test can continue its execution



Integration Testing

Kiểm thử tích hợp

- At the unit testing level, the system exists in pieces under the control of the programmers
- Put the modules together to construct the complete system
- The path from tested components to constructing a deliverable system contains two major testing phases:
 - **Integration testing**
 - **System testing**



Motivation of Integration Testing

7

- Different modules are generally created by groups of different developers
- Interface errors between modules created by different programmers and even by the same programmers are rampant
- Unit testing of individual modules is carried out in a controlled environment by using test drivers and stubs. It is therefore difficult to predict the behavior of a module in its actual environment after the unit testing is performed
- Some modules are more error prone than other modules, because of their inherent complexity. It is essential to identify the ones causing most failures.

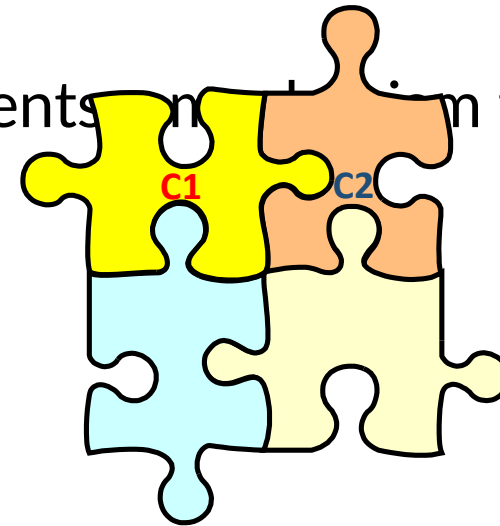
Objectives of Integration Testing

- To build a WORKING version of the system by putting the modules together
- To ensure that additional modules work as expected without disturbing the functionalities of the modules already put together
- Integration test is completed when the system is fully integrated together:
 - All the test cases have been executed
 - All the severe and moderate defects found have been fixed and the system is retested



Interfaces between modules

- Modules are interfaced with other modules to realize the system's functional requirements
- An interface between two modules allows one module to access the service provided by the other
- An interface between two modules implements mechanism for passing control and data between modules



Different types of Interfaces between modules

Three common paradigms for interfacing modules

- **Procedure Call Interface:** a procedure in one module call a procedure in another module. The caller passes on control and data to the called module. The called one can pass data to the caller while returning control back to the caller
- **Shared Memory Interface:** A block of memory is shared between two modules. Data are written into the memory block by one module and are read from the block by the other
- **Message Passing Interface:** One module prepares a message by intializing the fields of a data structure and sending the message to another module. This form is common in client-server or web-based system



Interface Errors

If all the unit tested modules work individually, why can these modules not work when put together?



Interface Errors

- **Construction:** interface specification is separated from the implementation code (i.e., in C/C++ through `#include header.h`). Inappropriate use of `#include` statements cause construction errors
- **Inadequate Functionality:** Implicit assumptions in one part of a system that another part of the system would perform a function
- **Location of Functionality:** Disagreement on or misunderstanding about the location of a functional capability within the software leads to this sort of error
- **Changes in Functionality:** Changing one module without correctly adjusting for that change in other related modules affects the functionality of the program



Interface Errors (cont.)

- **Added Functionality:** A completely new functional module was added as a system modification
- **Misuse of Interface:** one module makes an error in using the interface of a called module. Interface misuse can take the form of wrong parameter type, wrong parameter order or wrong number of parameters passed
- **Misunderstanding of Interface:** A calling module may misunderstand the interface specification of a called module. The called module may assume that some parameters passed to it satisfy a certain condition whereas the caller does not ensure that the condition holds.
- ...



Advantages of Integration Testing

- Defects are detected early
- It is easier to fix defects detected early
- Get earlier feedback on the health and acceptability of the individual modules and on the overall system
- Scheduling of defect fixes is flexible and it can overlap with development



Who do integration testing?

- System integration testing is performed by the system integration group, aka a build engineering group
 - Integration test engineers who should be familiar with the interface mechanisms
 - Team of engineers who built the modules
 - The system architects because of the fact that they have a bigger picture of the system



Granularity of System Integration Testing

- System integration testing is performed at different levels of granularity
- Use both of two techniques
 - Black box testing
 - White box testing
- Different levels:
 - Intrasytem Testing
 - Intersystem Testing
 - Pairwise Testing



Intrasystem Testing

- Low-level integration testing with the objective of combining the modules together to build a cohesive system
- Combining modules progresses in an incremental manner
- Example: in a client-server system, individual modules of client and server are combined in an incremental fashion, to be tested
- Source of testing: low-level design document which details the specification of modules



Intersystem Testing

- A high-level testing phase which requires interfacing independently tested systems
- All systems are connected together
- Testing is conducted from end to end: ensuring that the interaction between systems work together but not to conduct a comprehensive test
- Only one feature is tested at a time and on a limited basis
- Test cases are derived from the high-level design document, which details the overall system architecture
- Ex: integrating client and server to test after perform individually tested system (client and server)



Pairwise Testing

- There can be many intermediate levels of system integration testing between intrasystem testing and intersystem testing
- Pairwise testing is a kind of intermediate level of integration testing
- Only two interconnected systems in an overall system are tested at a time
- The purpose is to ensure that two systems under consideration can function together
- All other systems are assumed to behave as expected



System Integration Testing Techniques

Các kỹ thuật kiểm thử tích hợp hệ thống

- Incremental
- Top-down
- Bottom-up
- Sandwich
- Big bang



Incremental technique

- Integration testing is conducted in an incremental manner as a series of test cycles
- In each test cycle: a few more modules are integrated with an existing and tested build to generate a larger build
- Objective:
 - Complete one cycle of testing
 - Fix all the errors found
 - Continue the next cycle of testing
 - Finish building a complete and ready system for system testing



How many test cycles for system integration testing?

The number of system integration test cycles and the total integration time depend on:

- Number of modules in the system
- Relative complexity of the modules (McCabe Cyclomatic Complexity)
- Relative complexity of the interfaces between modules
- Number of modules needed to be clustered together in each test cycle
- Whether the modules to be integrated have been adequately tested before
- Turnaround time for each test-debug-fix cycle



Constructing a build in each test cycle

- Constructing a build is a process by which individual modules are integrated to form an interim **software image**
- A **software image** is a compiled software binary
- The **final build** will be a candidate for system testing
- Creating a daily build is very popular to facilitate a faster delivery of the system
- Focusing on small incremental testing, steadily increasing the number of test cases and regression testing from build to build





Software Quality Assurance and Testing

(end of lecture)

ONE LOVE. ONE FUTURE.